Praktikum
Object Oriented Program
2022

EAD Laboratory
EAD



Modul 2

Inheritance, Polimorfisme (Overriding, dan Overloading)

ASISTEN


ADIS


AFHM


ARTI


CACA


DAPA


DEYA


DIKA


DRFZ


FATH


FDIL


FRQI


HILM


NICA


OLLA


RAIN


SKUY


TITY


WANN


WISH


XASH

EAD Laboratory
EAD

Daftar Isi

Peraturan Praktikum **2**

Modul 2 Inheritance, Polimorfisme (Overriding, dan Overloading) **3**

Inheritance 3

Polimorfisme **4**

Overriding (Runtime Polymorphism) **7**

Contoh dari penggunaan method overriding 1: 8

Class Kendaraan 9

Class Mobil 10

Class Motor 11

Class Lambhorgini 12

Main Class 13

Output 14

Contoh dari penggunaan method overriding 2: 15

Output 15

Overloading (Compile time polymorphism) **16**

Contoh dari penggunaan method overloading : 17

Output: 19

Keuntungan Menggunakan Overloading Method : 20

DAFTAR PUSTAKA **21**

Peraturan Praktikum

1. Setiap peserta praktikum harus datang tepat waktu sesuai dengan jadwal. Toleransi keterlambatan hadir 10 menit. Jika melebihi batas waktu tersebut dan tidak dapat memberikan bukti alasan keterlambatan maka praktikan tidak diperkenankan mengikuti praktikum.

2. Perizinan Praktikum :
    a. Izin berkaitan dengan Sakit atau Kemalangan , maka praktikan dapat memberikan surat perizinan kepada pihak Komisi Disiplin maksimal 3 hari setelah jadwal (shift) praktikum.
    b. Izin lomba atau penugasan institusi tidak berlaku apabila tidak terdapat bukti dispensasi dari Igracias. NB : screenshot dispensasi dari igracias wajib dilampirkan.

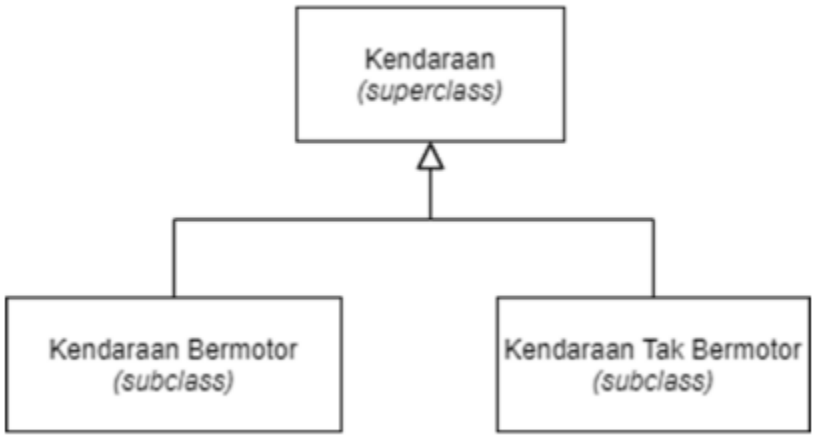
3. Seragam Praktikum :
    a. Mahasiswa wajib menggunakan celana bahan hitam (bukan chino atau jeans) pada saat praktikum.
    b. Mahasiswi wajib menggunakan rok hitam/biru gelap panjang tidak ketat pada saat praktikum
    c. Dresscode praktikum (Mengikuti Peraturan Telkom)
        - Senin : Menggunakan kemeja merah telkom atau kemeja putih polos
        - Selasa s/d Rabu : Menggunakan kemeja putih polos
        - Kamis s/d Sabtu : Menggunakan kemeja formal berkerah (bukan kerah sanghai dan bukan polo)
    d. Membuka sepatu saat memasuki ruangan lab.

Modul 2

Inheritance, Polimorfisme (Overriding, dan Overloading)

Inheritance

Inheritance atau pewarisan adalah salah satu konsep pada Object-Oriented Programming dimana sebuah Parent Class (superclass) mewariskan atribut dan method nya kepada Child Class (subclass). Atribut dan method yang diturunkan pada Child Class hanyalah yang memiliki modifier public dan protected.



Keterangan:

Class Kendaraan merupakan Parent Class(superclass) dari Class Kendaraan Bermotor dan Class Kendaraan Tak Bermotor, maka kedua Class tersebut memiliki ciri-ciri dan sifat yang mirip seperti Class kendaraan. Selain itu kedua Child Class(subclass) tersebut juga dapat memiliki ciri-ciri unik yang berbeda dari Class lainnya.

Polimorfisme

Polymorphism (Polimorfisme) berasal dari kata *poli* yang berarti "banyak" dan *morph* yang berarti bentuk sehingga secara sederhana polimorfisme merupakan kemampuan objek, variabel, atau fungsi yang dapat memiliki berbagai bentuk. Secara umum polymorphism dalam OOP terjadi ketika suatu SuperClass direferensikan ke dalam SubClass. Alhasil kita dapat mengembangkan sebuah program secara umum, bukan spesifik.

Polimorfisme dapat digambarkan seperti seorang yang menguasai dan mampu berbicara dalam beberapa bahasa. Nah di sini, seseorang bertindak sebagai obyek dan kemampuan dia berbicara merupakan sebuah polimorfisme. Contoh lainnya adalah smartphone. Selain sebagai alat komunikasi, smartphone yang bertindak sebagai objek dapat digunakan sebagai kamera, pemutar musik dan radio. Pada dasarnya, polimorfisme dapat dibagi menjadi dua, yaitu:

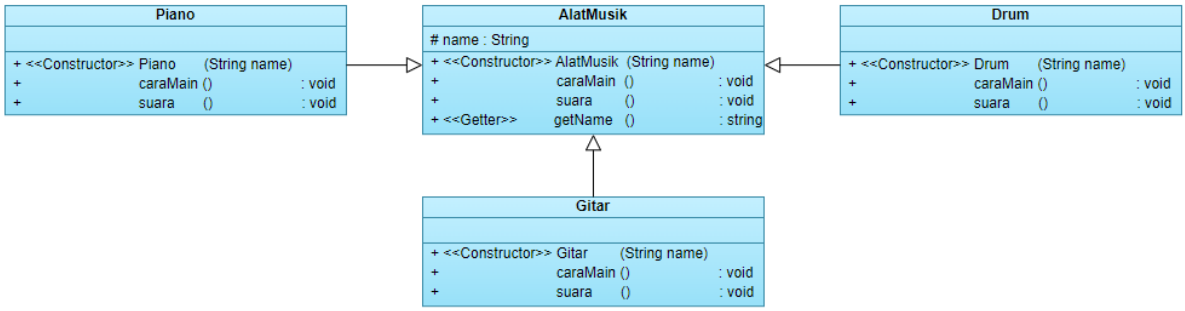
- Compile Time Polymorphism ***(Overloading)***
- Runtime Polymorphism ***(Overriding)***

Selain itu polimorfisme memiliki beberapa karakteristik, yaitu:

1. Dalam method overloading, jumlah dan tipe parameter serta urutannya atau dapat disebut dengan istilah method signature mengalami perubahan, sedangkan pada method overriding tidak berubah.
2. Method overloading dapat dilakukan pada class yang sama, namun method overriding hanya dapat dilakukan pada subclass dengan signature yang sama dan tipe return yang sama. Sedangkan pada method overloading, tipe return ini bisa sama atau berbeda.

3. Method private final atau static tidak dapat di override, namun method tersebut bisa overloading. Artinya suatu class dapat memiliki lebih dari satu method private final atau static dengan nama yang sama, namun subclass tidak dapat override private final atau static method tersebut dari super class.
4. Method overloading pada Java terikat oleh static binding sedangkan method overriding merupakan subjek dari dynamic binding.
5. Seperti pada keterangan jenis polimorfisme diatas method overloading terjadi pada saat compile time sedangkan overriding terjadi pada waktu runtime.

Berikut merupakan contoh penggambaran dari polimorfisme:



Method **caraMain()** dan suara() di class-class child dari class **AlatMusik** adalah contoh *overriding*. Berikut adalah contoh penerapan class diagram untuk *class* **AlatMusik** dan **Gitar**, dimana *method* **caraMain()** dan **suara()** di *class* **AlatMusik** di **Override** pada *class* **Gitar**.

```java
public class AlatMusik {
    protected String name;

    public AlatMusik (String name) { // constructor
        this.name = name
    }
    public String getname() {
        return name;
    }
    public void caraMain() {
        System.out.println("");
    }
    public void suara() {
        System.out.println("");
    }
}
```

```java
public class Gitar extends AlatMusik {
    public Gitar(String name) {
        super(name); // pewarisan atribut dari class parent
    }
    public void caraMain() {
        System.out.println("di petik");
    }
    public void suara() {
        System.out.println("jreng jreng jreng...");
    }
}
```

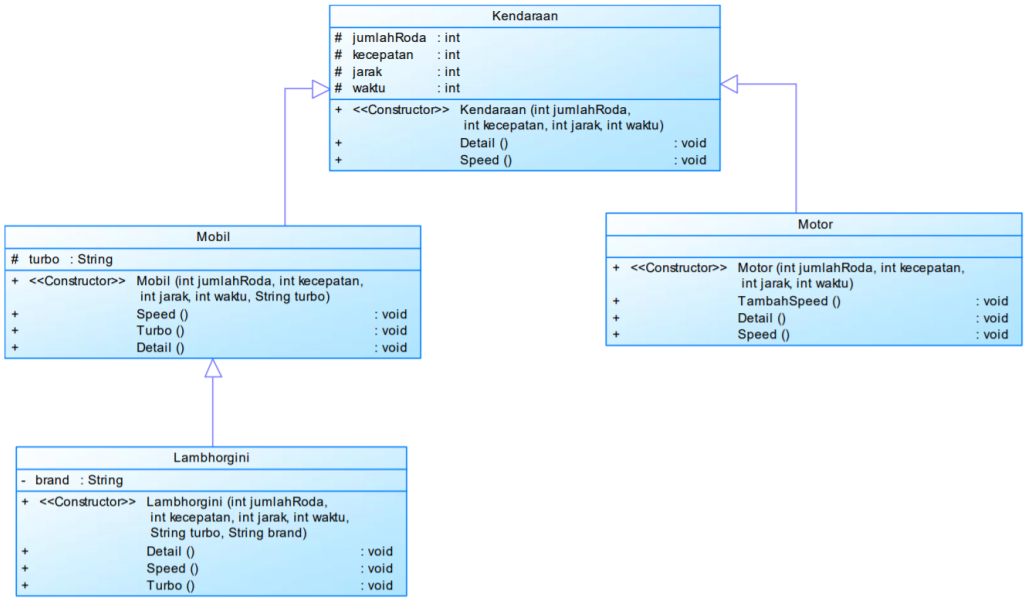
Overriding (Runtime Polymorphism)

Berbeda dengan *compile time* polimorfisme yang memanggil fungsi ketika proses kompilasi, runtime polimorfisme adalah proses di mana sebuah fungsi dipanggil pada saat runtime. Contoh dari runtime polimorfisme adalah *method overriding*, yaitu sebuah kelas yang memiliki fungsi dengan nama sama dengan fungsi yang di dalam kelas induknya.

Secara sederhana, *method overriding* adalah sebuah fitur pada sebuah bahasa pemrograman yang memungkinan *subclass* mewarisi sebuah implementasi yang spesifik dari sebuah fungsi yang ada pada kelas induknya (*parent class*), implementasi tersebut berupa method yang sama dengan parent *class*-nya namun memiliki implementasi yang berbeda. Implementasi pada *subclass* akan menimpa atau mengganti implementasi pada *parent class*. Berikut merupakan ciri-ciri dari method overriding:

1. Nama method dari child class harus sama dengan method di parent class.
2. Parameter dari method child class harus sama dengan method di parent class.
3. Return Type dari method child class harus sama dengan method di parent class.

Contoh dari penggunaan method overriding 1:



Terdapat 4 class seperti diatas yaitu Class Kendaraan, Class Mobil, Class Motor, dan Class Lamborghini. Berikut adalah rincian dari keempat kelas tersebut :

- Class Kendaraan
  - Terdapat 4 variabel protected yaitu jumlahRoda(int), kecepatan(int), jarak(int), waktu(int).
  - Satu constructor Kendaraan(int jumlahRoda, int kecepatan, int jarak, int waktu)
  - Dua Method public
- Class Mobil
  - Terdapat satu variabel protected yaitu turbo(String).
  - Satu constructor Mobil(int jumlahRoda, int kecepatan, int jarak, int waktu, String turbo)
  - Dua method override
  - Satu method public

- Class Motor
    - Satu constructor Motor(int jumlahRoda, int kecepatan, int jarak, int waktu)
    - Dua method override
    - Satu method public
- Class Lambhorgini
    - Terdapat satu variabel private yaitu brand(String).
    - Satu constructor Lambhorgini(int jumlahRoda, int kecepatan, int jarak, int waktu, String turbo, String brand)
    - Tiga method override

Class Kendaraan

```java
public class Kendaraan {
    protected int jumlahRoda, kecepatan, jarak, waktu; // attribute

    // constructor
    public Kendaraan(int jumlahRoda, int kecepatan, int jarak, int waktu){
        this.jumlahRoda = jumlahRoda;
        this.kecepatan = kecepatan;
        this.jarak = jarak;
        this.waktu = waktu;
    }

    // method void detail
    public void Detail(){
        System.out.println();
    }

    // method void speed
    public void Speed(){
        System.out.println();
    }
}
```

Class Mobil

```java
public class Mobil extends Kendaraan {
    protected String turbo; // attribute

    // constructor
    public Mobil(int jumlahRoda, int kecepatan, int jarak, int waktu, String turbo){
        super(jumlahRoda, kecepatan, jarak, waktu);
        this.turbo = turbo;
    }

    // overriding method detail dari class Kendaraan
    @Override
    public void Detail() {
        // TODO Auto-generated method stub
        System.out.println("Ini adalah kendaraan beroda " + jumlahRoda + " namun jenisnya belum diketahui");
        System.out.println("Jarak yang bisa ditempuh sejauh " + jarak + "km dengan waktu " + waktu + " jam");
    }

    // overriding method speed dari class Kendaraan
    @Override
    public void Speed() {
        // TODO Auto-generated method stub
        kecepatan = jarak * waktu;
        System.out.println("Setelah dihitung, mobil ini memiliki kecepatan sebesar " + kecepatan + "km/jam");
    }

    // method void turbo
    public void Turbo(){
        System.out.println("Mobil ini belum memiliki turbo");
    }
}

```

EAD Laboratory
EAD

Class Motor

```java
public class Motor extends Kendaraan {
    // constructor
    public Motor(int jumlahRoda, int kecepatan, int jarak, int waktu){
        super(jumlahRoda, kecepatan, jarak, waktu);
    }

    // overriding method detail dari class Kendaraan
    @Override
    public void Detail() {
        System.out.println("Ini adalah kendaraan beroda " + jumlahRoda + " namun jenisnya belum diketahui");
        System.out.println("Jarak yang bisa ditempuh sejauh " + jarak + "km dengan waktu " + waktu + " jam");
    }

    // overriding method speed dari class Kendaraan
    @Override
    public void Speed() {
        kecepatan = jarak * waktu;
        System.out.println("Setelah dihitung, motor ini memiliki kecepatan sebesar " + kecepatan + "km/jam");
    }

    // method void tambahSpeed
    public void tambahSpeed(int speedTambahan){
        kecepatan += speedTambahan;
        System.out.println("Setelah kecepatannya ditambah, kecepatannya menjadi " + kecepatan);
    }
}
```

Class Lambhorgini

```java
public class Lambhorgini extends Mobil {
    private String brand; // attribute

    // constructor
    public Lambhorgini(int jumlahRoda, int kecepatan, int jarak, int waktu, String turbo, String brand){
        super(jumlahRoda, kecepatan, jarak, waktu, turbo);
        this.brand = brand;
    }

    // overriding method detail dari class Mobil
    @Override
    public void Detail() {
        // TODO Auto-generated method stub
        System.out.println("Ini adalah kendaraan beroda " + jumlahRoda + " jenis " + "" + brand);
        System.out.println("Jarak yang bisa ditempuh sejauh " + jarak + "km dengan waktu " + waktu + " jam");
    }

    // overriding method speed dari class Mobil
    @Override
    public void Speed() {
        // TODO Auto-generated method stub
        super.Speed();
    }

    // overriding method turbo dari class Mobil
    @Override
    public void Turbo() {
        // TODO Auto-generated method stub
        System.out.println("Mobil ini sudah memiliki turbo");
    }
}

```

EAD Laboratory
EAD

Main Class

```java
public class Main {
    // main class
    public static void main(String[] args) throws Exception {
        System.out.println("Class Mobil");
        // pembuatan objek mobil1 dari class Mobil
        Mobil mobil1 = new Mobil(4, 0, 60, 2, "");
        mobil1.Detail(); // pemanggilan method detail objek mobil1
        mobil1.Speed(); // pemanggilan method speed objek mobil1
        mobil1.Turbo(); // // pemanggilan method turbo objek mobil1
        System.out.println("");

        System.out.println("Class Motor");
        // pembuatan objek motor1 dari class Motor
        Motor motor1 = new Motor(2, 0, 30, 1);
        motor1.Detail(); // pemanggilan method detail objek motor1
        motor1.Speed(); // pemanggilan method speed objek motor1
        motor1.tambahSpeed(25); // pemanggilan method tambahSpeed objek motor1
        System.out.println("");

        System.out.println("Class Lamhorgini");
        // pembuatan objek lambo dari class Lambhorgini
        Lambhorgini lambo = new Lambhorgini(4, 0, 100, 1, "", "Aventador");
        lambo.Detail(); // pemanggilan method detail objek lambo
        lambo.Speed(); // pemanggilan method speed objek lambo
        lambo.Turbo(); // pemanggilan method turbo objek lambo
    }
}
```

Output

```
Class Mobil
Ini adalah kendaraan beroda 4 namun jenisnya belum diketahui
Jarak yang bisa ditempuh sejauh 60km dengan waktu 2 jam
Setelah dihitung, mobil ini memiliki kecepatan sebesar 120km/jam
Mobil ini belum memiliki turbo

Class Motor
Ini adalah kendaraan beroda 2 namun jenisnya belum diketahui
Jarak yang bisa ditempuh sejauh 30km dengan waktu 1 jam
Setelah dihitung, motor ini memiliki kecepatan sebesar 30km/jam
Setelah kecepatannya ditambah, kecepatannya menjadi 55

Class Lamhorgini
Ini adalah kendaraan beroda 4 jenis Aventador
Jarak yang bisa ditempuh sejauh 100km dengan waktu 1 jam
Setelah dihitung, mobil ini memiliki kecepatan sebesar 100km/jam
Mobil ini sudah memiliki turbo
```

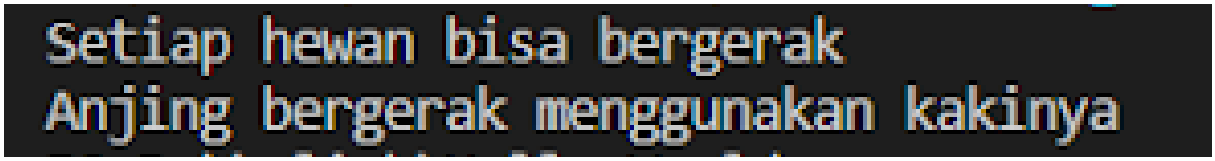
Contoh dari penggunaan method overriding 2:

```java
class Hewan {
    public void move() {
        System.out.println("Setiap hewan bisa bergerak");
    }
}

class Anjing extends Hewan {
    @Override
    public void move() {
        // TODO Auto-generated method stub
        super.move();
        System.out.println("Anjing bergerak menggunakan kakinya");
    }
}

public class Testing {
    public static void main(String args[]) {
        Anjing anjing = new Anjing();
        anjing.move();
    }
}
```

Output



Overloading (Compile time polymorphism)

Compile time polymorphism adalah sebuah proses di mana sebuah method atau fungsi dipanggil saat kompilasi. Ini dapat terjadi karena sebuah konsep bernama method overloading.

Method overloading artinya method dengan nama yang sama, namun memiliki parameter yang berbeda, dan method ini berada dalam sebuah class yang sama atau bisa juga berada dalam class yang lain yang terkait dalam hirarki inheritance.

Adapun cara kerja dari method overloading, antara lain :

1. Method overloading harus memiliki nama yang sama seperti parent classnya.
2. Method overloading harus memiliki parameter yang berbeda dengan parent classnya.
3. Dapat dilakukan dengan mengubah jumlah, urutan dan atau tipe data parameternya.

Berikut apabila method overloading dapat diterima :

- Kuantitas Parameter : Method Overloading dapat diterima di dalam class mengingat jumlah/kuantitas parameter tidak sama. Contoh :

```
public void luas(double p, double l){
    System.out.println(p*l);
}

public void luas(double p, double l, double t){
    System.out.println(p*l*t);
}
```

- Tipe Data Parameter : Method Overloading dapat diterima di dalam class setidaknya satu dari setiap method overloading memiliki tipe data yang berbeda. Contoh :

```
public void luas(double p, double l){
    System.out.println(p*l);
}

public void luas(int p, int l){
    System.out.println(p*l);
}
```

- Urutan Parameter : Method Overloading dapat diterima di dalam sebuah class ketika urutan variabel tipe data tidak sama. Contoh :

```
public void luas(double p, double l){
    System.out.println(p*l);
}

public void luas(int l, int p){
    System.out.println(p*l);
}
```

- Return type : Method overloading dapat diterima walaupun return type nya berbeda. Contoh :

```
public void luas(double p, double l){
    System.out.println(p*l);
}

public int luas(int l, int p){
    return p*l;
}
```

Contoh dari penggunaan method overloading :

Contoh penerapannya terbagi atas 2 (dua) yang didasari atas pengertian method overloading itu sendiri. Pertama adalah parameter type dengan contoh kode seperti di bawah ini:

```java
public class App {
    public Double luas(double p, double l){
        return p*l;
    }

    public int luas(int l, int p){
        return p*l;
    }

    public Double luas(long p, double l){
        return p*l;
    }

    public long luas(long p, int l){
        return p*l;
    }
}
```

Method overloading di atas dibedakan atas tipe argumen yang menjadi parameter dari fungsi tersebut. Bagaimana pemanggilan dari beberapa fungsi di atas ditentukan? Kuncinya ada pada argumen yang disematkan ketika fungsi tersebut digunakan di dalam program. Argumen tersebut mendasari berjalannya proses kompilasi.

Kemudian, jumlah parameter sebuah fungsi yang dapat memiliki **nama yang sama** namun **jumlah parameter yang berbeda** seperti berikut:

```java
public class App {
    public Double luas(double p, double l){
        return p*l;
    }

    public int luas(int l, int p){
        return p*l;
    }

    public Double luas(long p, double l){
        return p*l;
    }

    public long luas(long p, int l){
        return p*l;
    }

    public long luas(long p, int l, int t){
        return p*l*t;
    }
}
```

```java
public class MainApp extends App{
    Run | Debug
    public static void main(String[] args) {
        long panjang = 10;
        int lebar = 10;
        int tinggi = 10;

        App app = new App();
        System.out.println("Luas 1 yaitu : " + app.luas(panjang, lebar));
        System.out.println("Luas 2 yaitu : " + app.luas(panjang, lebar, tinggi));
    }
}
```

Output:

```
Luas 1 yaitu : 100
Luas 2 yaitu : 1000
```

Keuntungan Menggunakan Overloading Method :

- Dapat membuat banyak method dengan nama sama.
- Apabila tidak menggunakan overloading method, maka harus dibuatkan banyak method. Contoh : maks1 dengan 2 parameter, maks2 untuk 3 parameter.
- Overloading juga dapat digunakan pada constructor

EAD Laboratory
EAD

DAFTAR PUSTAKA

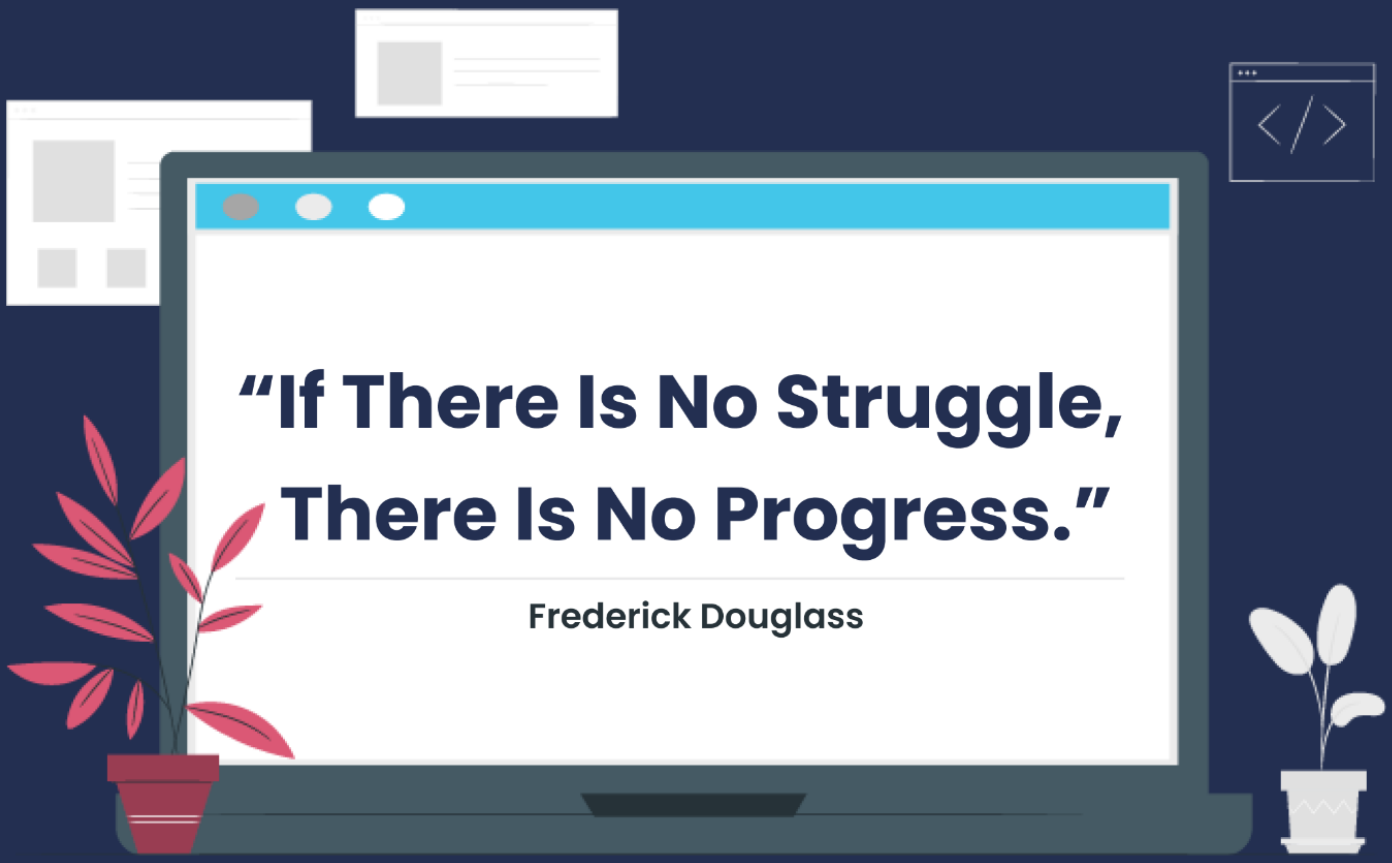
[1] A. (2021, September 12). Tutorial Java 17 : Mengenal Overriding Java dan Superclass Subclass | CODEKEY. CODEKEY |. Retrieved December 23, 2021, from https://codekey.id/java/overriding-java

[2] Indonesia, D. (2021). Dicoding Indonesia. Dicoding. Retrieved December 23, 2021, from https://www.dicoding.com/academies/169/tutorials/7788

[3] Modul 3 Polymorphisme, Overloading, dan Overriding. (2019). EAD Laboratory.

[4] Muhardian, A. (2021, December 22). Belajar Java OOP: Memahami Inheritance dan Method Overriding. Petani Kode. Retrieved December 23, 2021, from https://www.petanikode.com/java-oop-inheritance

EAD Laboratory
EAD



Find us on :

@eadlaboratory

@XXX